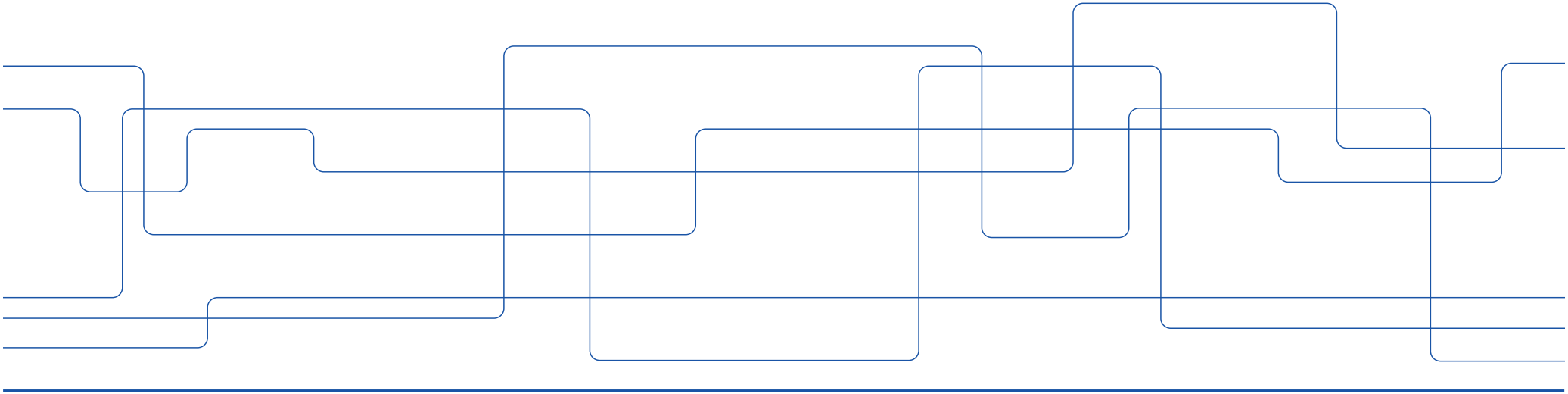


HI1024

Grundkurs i programmering

F5: Fördjupning enkla datatyper!



Innehåll!

King 7

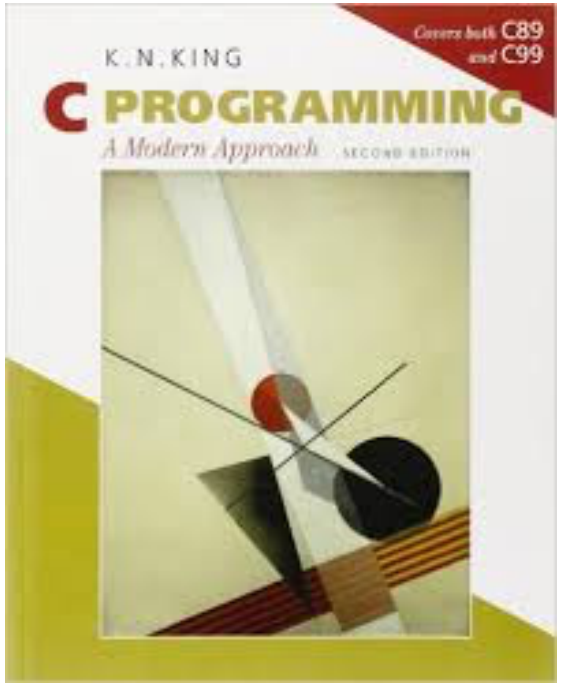
Repetition föreläsning #4

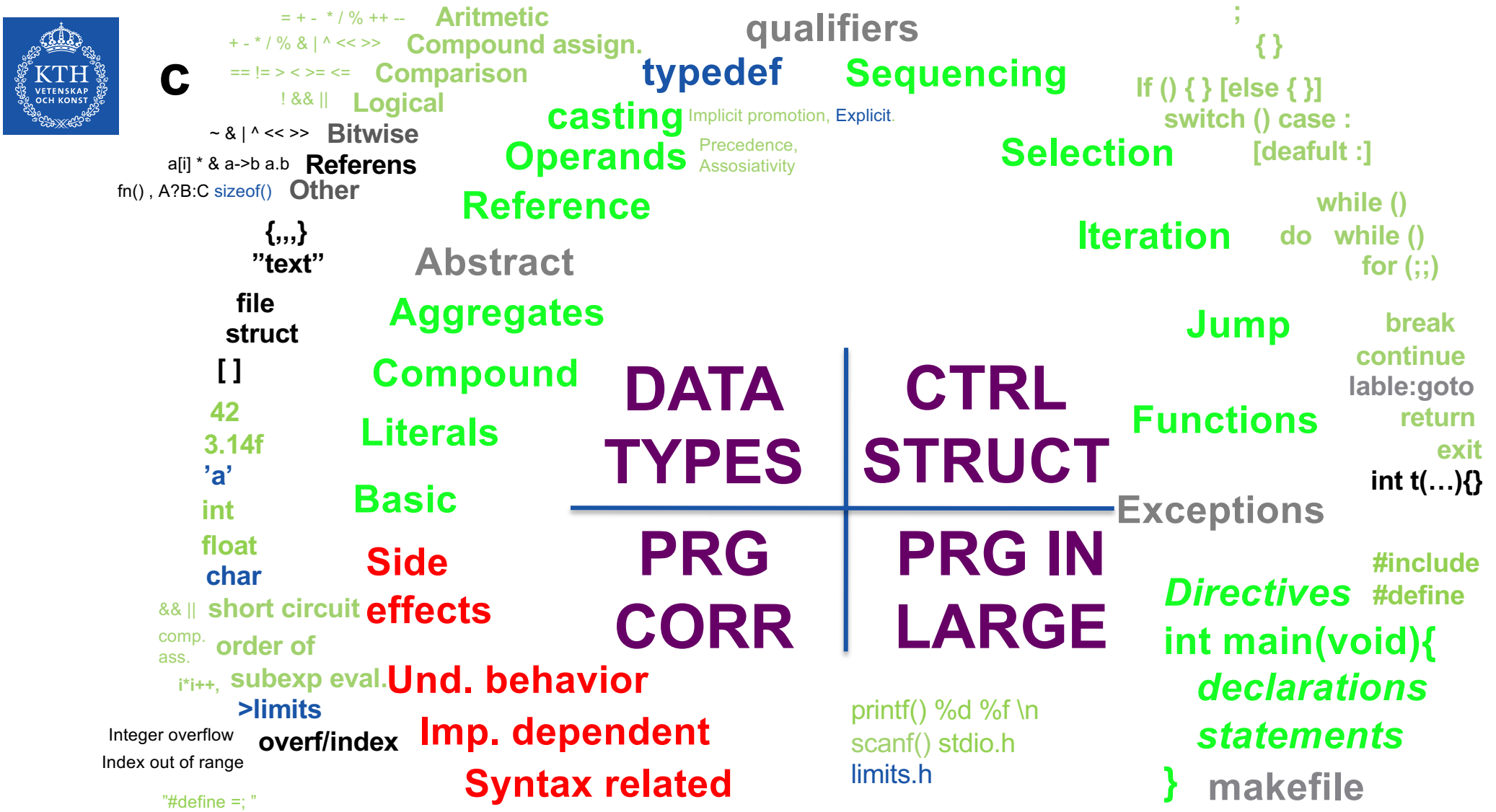
limits.h

Explict typkonvertering

char

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
&& ||
short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
int/int
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
Reference
Abstract
Aggregates
Compound
Literals
Basic
Side effects
Und. behavior
Imp. dependent
Syntax related
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
printf() %d %f \n
scanf() stdio.h
limits.h
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
#include
#define
Directives
int main(void){
declarations
statements
} makefile
/*comment*/ or //

Tangentbordsbufferten…



scanf(“%d”, &nb);

OS:ets tangentbordsbuffert: “ <tom> “

Vad händer?

Tangentbordsbufferten…



scanf(“%d”, &nb);

OS:ets tangentbordsbuffert: “ <tom> “

Vad händer? Programmet “stannar”!
(**Inget** händer förrän du trycker return!)
Du skriver: 123 47 HEJ! <RETURN>

Vad händer?

Tangentbordsbufferten…



scanf(“%d”, &nb);

OS:ets tangentbordsbuffert: “ <tom> “

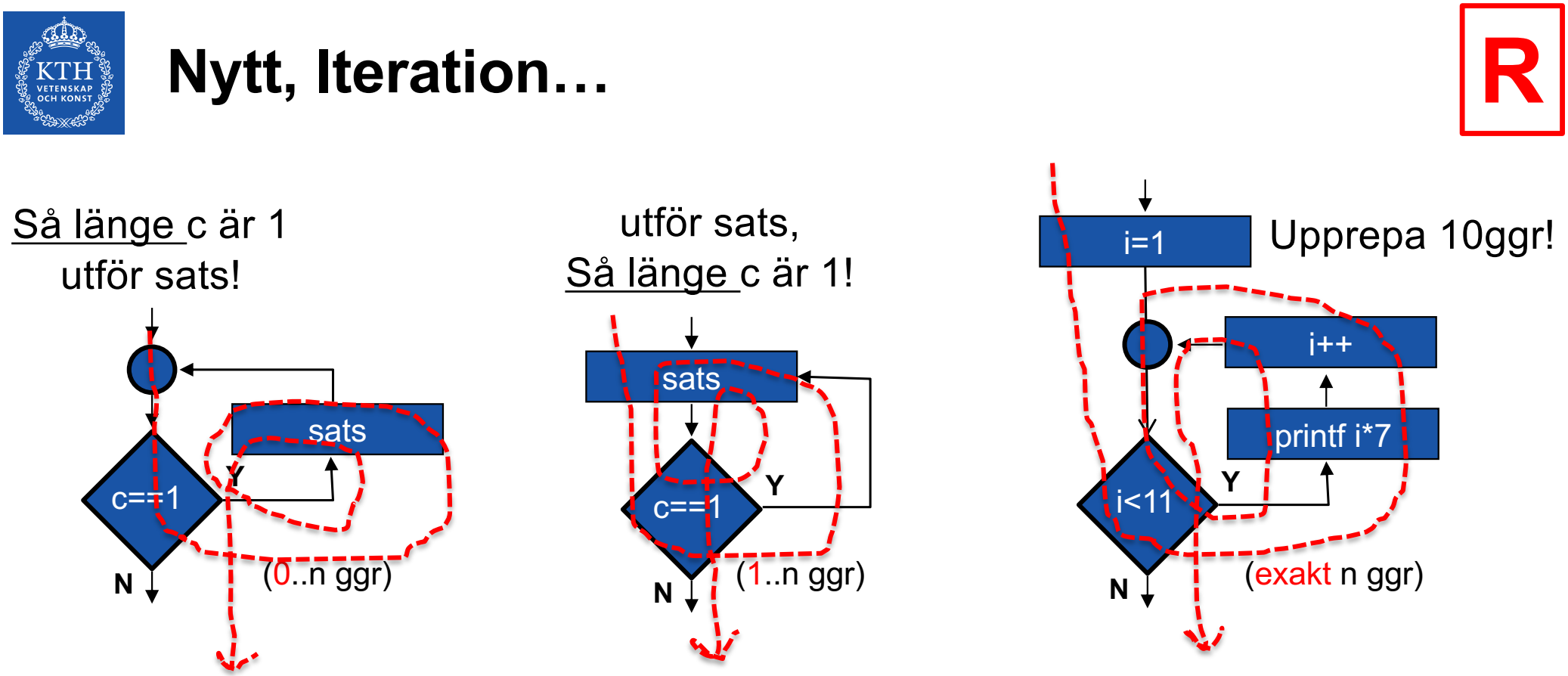
Vad händer? Programmet “stannar”!
(**Inget** händer förrän du trycker return!)
Du skriver: 123 47 HEJ! <RETURN>



Vad händer? nb får värdet 123 **och**

OS:ets tangentbordsbuffert: “ 47 HEJ! “

Nytt, Iteration…



while (expression)
statement

do
statement
while (expression);

initiering test modifiering
for (expr1 ; expr2 ; expr3)
statement

Som med if får statement bara vara en sats, i annat fall måste den vara sammansatt!

Nästlade iterationer…

```c
/* Iterators 0.7 AC */
#include <stdio.h>
#define SIZE 4

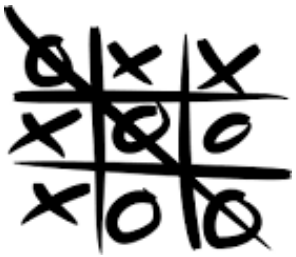
int main(void){

    printf("Iteration Tester (IT)!\n");

    for (int i=0;i<SIZE;i++) {
        for (int j=0;j<SIZE;j++)
            printf("(%d,%d)",i,j);
        printf("\n");
    }

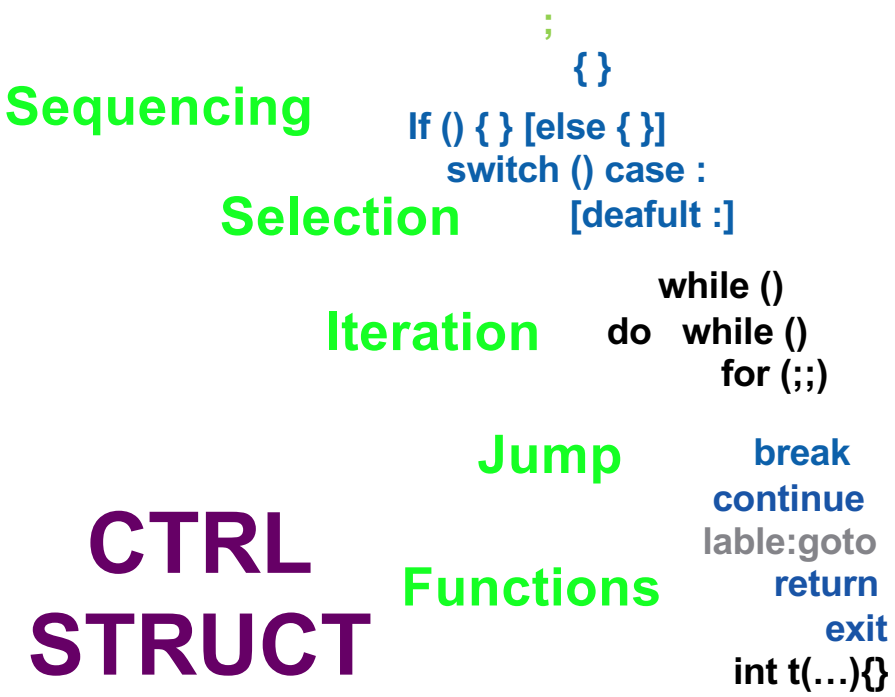
    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)!
(0,0)(0,1)(0,2)(0,3)
(1,0)(1,1)(1,2)(1,3)
(2,0)(2,1)(2,2)(2,3)
(3,0)(3,1)(3,2)(3,3)
IT Done.
```



Tänk på att for-satserna måste ha olika loop-variabler!!!

Hopp…



Break avslutar en while, do, for-loop eller en switch case-sats. God programmeringssed tillåter dylikt!

Continue hoppar över återstående satser i en iteration men avslutar inte iterationen. Används sparsamt!

Goto hoppar till en angiven plats i koden. Vi använder **inte** goto i kursen och det används ytterst sällan, möjligen för att hantera sällsynta fel, ute i industrin.

Return avslutar en funktion, och finns det bara en (main) så avslutar det programmet. Måste användas!

Exit avslutar programmet, men vi använder return.

Övning #4: Reflektioner…

Max of 0..n Numbers

Best practice aka Design pattern

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int m=0;

    printf("Max Finder (MF)\n");

    for (int n=-1; n!=0;) {
        printf("Enter a nb, exit with 0 : ");
        scanf("%d", &n);
        if (n>m) m=n;
    }

    printf("Max entry was %d !\n", m);

    printf("MF Done.\n");
    return 0;
}
```

```
Max Finder (MF)
Enter a nb, exit with 0 : 1
Enter a nb, exit with 0 : 3
Enter a nb, exit with 0 : 2
Enter a nb, exit with 0 : 0
Max entry was 3 !
MF Done.
```

Bra

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int m=0, n=-1;

    printf("Max Finder (MF)\n");

    while (n!=0) {
        printf("Enter a nb, exit with 0 : ");
        scanf("%d", &n);
        if (n>m) m=n;
    }

    printf("Max entry was %d !\n", m);

    printf("MF Done.\n");
    return 0;
}
```

```
Max Finder (MF)
Enter a nb, exit with 0 : 1
Enter a nb, exit with 0 : 3
Enter a nb, exit with 0 : 2
Enter a nb, exit with 0 : 0
Max entry was 3 !
MF Done.
```

Bättre

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int m=0, n;

    printf("Max Finder (MF)\n");

    do {
        printf("Enter a nb, exit with 0 : ");
        scanf("%d", &n);
        if (n>m) m=n;
    } while (n!=0);

    printf("Max entry was %d !\n", m);

    printf("MF Done.\n");
    return 0;
}
```

```
Max Finder (MF)
Enter a nb, exit with 0 : 1
Enter a nb, exit with 0 : 3
Enter a nb, exit with 0 : 2
Enter a nb, exit with 0 : 0
Max entry was 3 !
MF Done.
```

Bäst

Typfall där vi vet att vi måste utföra loopen minst 1 gång, dock inte hur många ggr, det är då ofta bäst att använda do!

Övning #4: Reflektioner

```c
#include <stdio.h>

int main(void){
    int nbOfDays=31;
    printf("Month calendar\n");

    for (int i=0; i<nbOfDays; i++) {
        printf("%2d ",i+1);

        if ((i+1)%7==0) {
           printf("\n");
        }
    }

    printf("\n");
    printf("Done-bye!\n");
}
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
Month calendar
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
Done-bye!
n163-p111:vsws ac$
```

Om vi nu vill att månadens första dag är en onsdag (3)…

…betyder det inte att vi vill göra exakt samma sak, men vi måste skriva ut 2 dagar till – där de två första är “ “?!

Övning #4: Reflektioner

```c
#include <stdio.h>

int main(void){
    int nbOfDays=31;
    printf("Month calendar\n");

    for (int i=0; i<nbOfDays; i++) {
        printf("%2d ",i+1);

        if ((i+1)%7==0) {
           printf("\n");
        }
    }

    printf("\n");
    printf("Done-bye!\n");
}
```

Om vi nu vill att månadens första dag är en onsdag (3)…

…betyder det inte att vi vill göra exakt samma sak, men vi måste skriva ut 2 dagar till – där de två första är “ “?!

```
OUTPUT  TERMINAL  DEBUG CONSOLE  PROBLEMS

Month calendar
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
Done-bye!
n163-p111:vsws ac$
```

```c
#include <stdio.h>

int main(void){
    int nbOfDays=31;
    int firstDay=3;
    printf("Month calendar\n");

    for (int i=0; i<nbOfDays+firstDay-1; i++) {
        if (i<(firstDay-1))
          printf("   ");
        else
          printf("%2d ",i-firstDay+2);

        if ((i+1)%7==0) {
          printf("\n");
        }
    }

    printf("\n");
    printf("Done-bye!\n");
}
```

```
OUTPUT  TERMINAL  DEBUG CONSOLE  PROBLEMS

Month calendar
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
Done-bye!
n163-p111:vsws ac$
```



Bergis?!

?!

```
printf( ”Result %d, value” );

if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```



Bergis?!



printf(”Result %d, value”);

printf(”Result %d”, value);

```
if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```


Det är enbart den första parametern som ska vara inom citattecken, i annat fall blir det väldigt fel!



Bergis?!

?!

```
printf( ”Result %d, value” );
```

```
printf( ”Result %d”, value );
```

```
if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```

Bergis?!

```
printf( ”Result %d, value” );
```



```
if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```

```
printf( ”Result %d”, value );

if (x>7) {
   printf( ”Svaret är\n” );
   printf( ”större än 7\n” ); }
```


Indenteringen lurar för en gång skull ögat att tro att bägge satserna tillhör if-satsen, vilket är fel!



Bergis?!

```
printf( ”Result %d, value” );

if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```

?!

```
printf( ”Result %d”, value );

if (x>7) {
   printf( ”Svaret är\n” );
   printf( ”större än 7\n” ); }
```

Bergis?!

```
printf( ”Result %d, value” );

if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```

```
printf( ”Result %d”, value );

if (x>7) {
   printf( ”Svaret är\n” );
   printf( ”större än 7\n” ); }

for (int i=0; i<10; i++)
   printf( ” i = %d ”, i);
```

Ett ensamt semikolon är en sats i c, vilket avslutar loopen, och gör att i inte längre finns på nästa rad!



Bergis?!

```
printf( ”Result %d, value” );

if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
   b=2;
}; else {
   b=3;
};
```

?!

```
printf( ”Result %d”, value );

if (x>7) {
   printf( ”Svaret är\n” );
   printf( ”större än 7\n” ); }

for (int i=0; i<10; i++)
   printf( ” i = %d ”, i);
```

Bergis?!

```
printf( ”Result %d, value” );

if (x>7)
  printf( ”Svaret är\n” );
  printf( ”större än 7\n” );

for (int i=0; i<10; i++);
  printf( ” i = %d ”, i);

If (a=2) {
  b=2;
}; else {
  b=3;
};
```

```
printf( ”Result %d”, value );

if (x>7) {
   printf( ”Svaret är\n” );
   printf( ”större än 7\n” ); }

for (int i=0; i<10; i++)
   printf( ” i = %d ”, i);

If (a==2) {
   b=2;
} else {
   b=3;
}
```


Flera fel: a=2 betyder att a tilldelas 2 med värdet 2 som är skilt från 0 = sant & felaktiga semikolon!

Nytt: limits.h

```c
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    int n;
    int i=1;

    printf("Iteration Tester (IT)!\n");

    printf("Ange ett tal :");
    scanf("%d",&n);

    while (n>10) {
        n=n/10;
        i++;
    }

    printf("Talet bestod av %d siffror!\n",i);

    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)
Ange ett tal : 1
Talet bestod av 1 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 123456789
Talet bestod av 9 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 1234567890
Talet bestod av 10 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 12345678901
Talet bestod av 1 siffror!
IT Done.
```

Eh, vad händer här?!

Nytt: limits.h

```c
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    int n;
    int i=1;

    printf("Iteration Tester (IT)!\n");

    printf("Ange ett tal :");
    scanf("%d",&n);

    while (n>10) {
        n=n/10;
        i++;
    }

    printf("Talet bestod av %d siffror!\n",i);

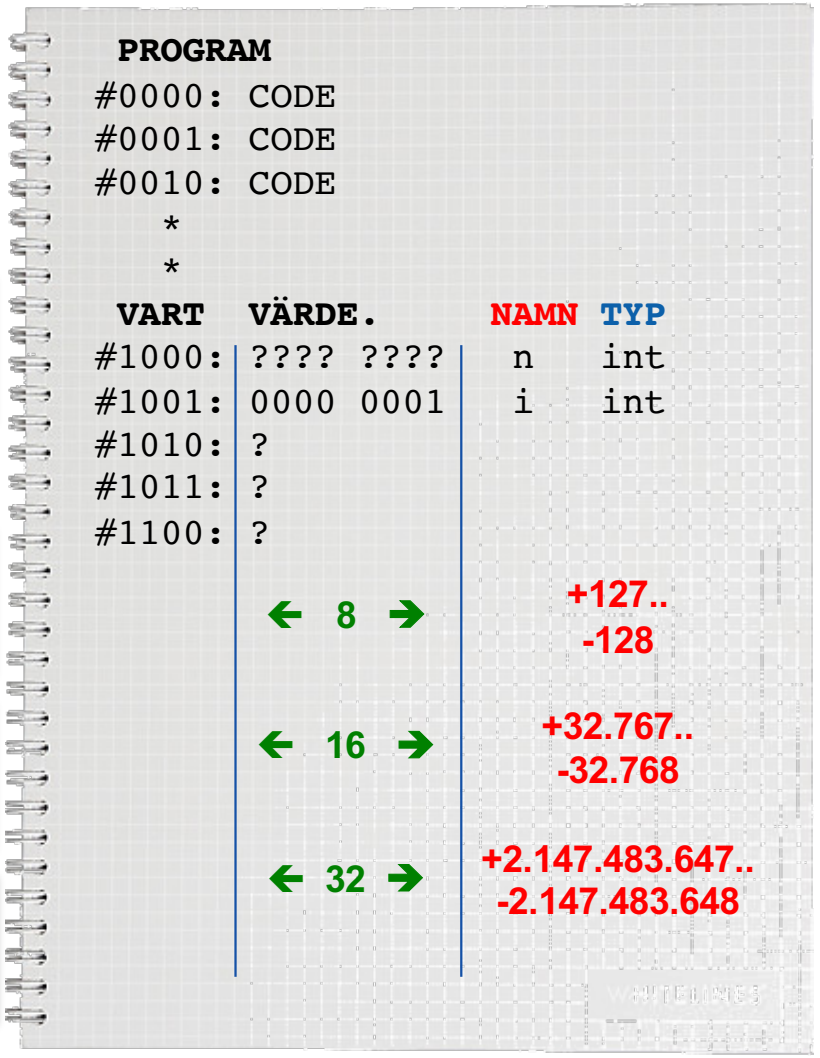
    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)
Ange ett tal : 1
Talet bestod av 1 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 123456789
Talet bestod av 9 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 1234567890
Talet bestod av 10 siffror!
IT Done.
```

```
Iteration Tester (IT)
Ange ett tal : 12345678901
Talet bestod av 1 siffror!
IT Done.
```



Undefined behavior!

Nytt: limits.h

Självklart går det att skapa utrymme för stora tal på en 8 eller 16 bitars maskin och jättestora tal på en 32-bitars…

…kompilatorn tar helt enkelt flera adresser i anspråk för att lagra en variabel…

…sen måste minnescellerna fogas samman, på rätt sätt, till en fiktiv stor minnescell varje gång variabeln används!

…den minst värdefulla byten kallas least significant byte och den mest värdefulla most significant byte!

```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN TYP
#1000: ???? ????      n   int
#1001: ???? ????
#1010: ???? ????
#1011: ???? ????
#1100: 0000 0001      i   int LSB
#1101: 0000 0000
#1110: 0000 0000
#1111: 0000 0000              MSB
```

Tyvärr så arbetar en del processorer med LSB/W/DW på den lägsta adressen (Little endian) (intel) och andra med MSB/W/DW på den lägsta adressen (Big endian)(IBM) vilket är ett av skälen till att kod sällan kan flyttas mellan olika processorfamiljer.



Nytt: limits.h

Typ	minsta värde	största värde	printf,scanf
short int	-32 768	32 767	%hd
unsigned short int	0	65 535	%hu
int	-2 147 483 648	2 147 483 647	%d
unsigned int	0	4 294 967 295	%u
long int	-2 147 483 648	2 147 483 647	%ld
unsigned long int	0	4 294 967 295	%lu

Typ	Minsta positiva värde	Största värde	Presicion	printf, scanf
float	1.17549e-38	3.40282e38	6 siffror	%f, %f
double	2.22507e-308	1.79769e308	15 siffror	%f, %lf

(unsigned long long int 18,446,744,073,709,551,615 är det största exakta talet som finns)

Nytt: Explicit typkonvertering

(typ) – denna operator konverterar om möjligt talet till höger till datatypen typ.



```
int i = (int) 3.45;
```

Konverteringen sker dock automatiskt vid behov:

```
int i = 3.45;//ger samma resultat

float f = 3 / 2;          (HL int därefter implicit konv. till float)   //f blir 1.0
f = 3.0 / 2;              (HL float)                                    //f blir 1.5

int i1 = 3;
int i2 =2;
f = i1 / i2;              (HL int därefter implicit konv. till float)   //f blir 1.0
f = (float) i1 / i2;      (HL float, tack vare explicit konv. av i1))   //f blir 1.5
```



Övning, nästlade loopar

- Skriv ut kortlek snyggt

```
Deck of cards 0.1:
Club    a
Club    2
Club    3
Club    4
Club    5
Club    6
Club    7
Club    8
Club    9
Club    t
Club    j
Club    q
Club    k
Diamond a
Diamond 2
Diamond 3
Diamond 4
Diamond 5
Diamond 6
Diamond 7
Diamond 8
Diamond 9
Diamond t
Diamond j
Diamond q
Diamond k
Heart   a
Heart   2
Heart   3
Heart   4
Heart   5
Heart   6
Heart   7
Heart   8
Heart   9
Heart   t
Heart   j
Heart   q
Heart   k
Spade   a
Spade   2
Spade   3
Spade   4
Spade   5
Spade   6
Spade   7
Spade   8
Spade   9
Spade   t
Spade   j
Spade   q
Spade   k
Deck of cards done.
```

1 Skriv ner proble-met så att ett barn förstår det!

Tips: Fyra färger, med 13 kort i varje färg…



Övning, nästlade loopar

- Skriv ut kortlek snyggt

För alla färger…
För alla valörer…
Skriv ut färg & valör!

1 **Skriv ner proble-met så att ett barn förstår det!** **2** **3**

```
Deck of cards 0.1:
Club    a
Club    2
Club    3
Club    4
Club    5
Club    6
Club    7
Club    8
Club    9
Club    t
Club    j
Club    q
Club    k
Diamond a
Diamond 2
Diamond 3
Diamond 4
Diamond 5
Diamond 6
Diamond 7
Diamond 8
Diamond 9
Diamond t
Diamond j
Diamond q
Diamond k
Heart   a
Heart   2
Heart   3
Heart   4
Heart   5
Heart   6
Heart   7
Heart   8
Heart   9
Heart   t
Heart   j
Heart   q
Heart   k
Spade   a
Spade   2
Spade   3
Spade   4
Spade   5
Spade   6
Spade   7
Spade   8
Spade   9
Spade   t
Spade   j
Spade   q
Spade   k
Deck of cards done.
```

Ö ar

- S

```c
// Deck of cards 0.1
#include <stdio.h>
#define NBOFSUITS 4
#define CARDSINSUIT 13

int main(void){

   printf("Deck of cards 0.1:\n");

   for (int suit=0; suit<NBOFSUITS; suit++) {
      for (int rank=0; rank<CARDSINSUIT; rank++) {
        switch (suit) {
           case 0 : printf("Club    "); break;
           case 1 : printf("Diamond "); break;
           case 2 : printf("Heart   "); break;
           case 3 : printf("Spade   "); break;
        }
        switch (rank) {
           case 0  : printf("a\n"); break;
           case 1  : printf("2\n"); break;
           *
           case 8  : printf("9\n"); break;
           case 9  : printf("t\n"); break;
           case 10 : printf("j\n"); break;
           case 11 : printf("q\n"); break;
           case 12 : printf("k\n"); break;
        }
      }
   }
   printf("Deck of cards done.\n");
   return 0;
}
```

```
Deck of cards 0.1:
Club    a
Club    2
Club    3
Club    4
Club    5
Club    6
Club    7
Club    8
Club    9
Club    t
Club    j
Club    q
Club    k
Diamond a
Diamond 2
Diamond 3
Diamond 4
Diamond 5
Diamond 6
Diamond 7
Diamond 8
Diamond 9
Diamond t
Diamond j
Diamond q
Diamond k
Heart   a
Heart   2
Heart   3
Heart   4
Heart   5
Heart   6
Heart   7
Heart   8
Heart   9
Heart   t
Heart   j
Heart   q
Heart   k
Spade   a
Spade   2
Spade   3
Spade   4
Spade   5
Spade   6
Spade   7
Spade   8
Spade   9
Spade   t
Spade   j
Spade   q
Spade   k
Deck of cards done.
```

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	*warning: extra tokens at end of #include directive*

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	*warning: extra tokens at end of #include directive*
6	error: expected ';' after expression

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	*warning: extra tokens at end of #include directive*
6	error: expected ';' after expression
7	error: use of undeclared identifier 'r1'

+ Dolt fel: Adressoperatorn & saknas!!!

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	*warning: extra tokens at end of #include directive*
6	error: expected ';' after expression
7	error: use of undeclared identifier 'r1'
8	error: expected ';' after expression
9	error: use of undeclared identifier 'r2'

+ Dolt fel: Adressoperatorn & saknas!!!

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	*warning: extra tokens at end of #include directive*
6	error: expected ';' after expression
7	error: use of undeclared identifier 'r1'
8	error: expected ';' after expression
9	error: use of undeclared identifier 'r2'
11	error: use of undeclared identifier 'rx'
11	error: use of undeclared identifier 'r1'
11	error: use of undeclared identifier 'r2'
11	error: use of undeclared identifier 'r1'
11	error: use of undeclared identifier 'r2'

Se upp med prioritet!!!

Vad säger kompilatorn?

```
1   #include <stdio.h>;
2
3   int main(void){
4
5
6     printf("Enter first value: ")
7     scanf("%d",r1);
8     printf("Enter second value: ")
9     scanf("%d",r2);
10
11    rx=r1*r2/(r1+r2);
12    printf("Replace with %d ohm!\n");
13  }
```

Line	Message
1	warning: extra tokens at end of #include directive
6	error: expected ';' after expression
7	error: use of undeclared identifier 'r1'
8	error: expected ';' after expression
9	error: use of undeclared identifier 'r2'
11	error: use of undeclared identifier 'rx'
11	error: use of undeclared identifier 'r1'
11	error: use of undeclared identifier 'r2'
11	error: use of undeclared identifier 'r1'
11	error: use of undeclared identifier 'r2'
12	warning: more '%' conversions than data arguments [

Taktik: Kompilera ofta = få nya rader = få fel. Rätta ett fel, kan ta bort följdfel!

Nytt: Datatypen char

- **Som ni säkert förstår vid det här laget går det bara att lagra tal i en dators minne…**
- …de går definitivt inte att lagra en bokstav, t.ex ’a’, än mindre ett ord…
- **…men vi skulle kunna lagra vilken position en specifik bokstav har i ett specifikt alfabet, om vi vet att den minnescellen förväntas lagra en bokstav!**

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: ???? ????     n     int
#1001: 0000 0001     i     int
#1010: ?
#1011: ?
#1100: 0001 0100     tkn   bokstav
```

Aha! Ett ’T’

1.	2.	3																	20									
Versaler (*"stora bokstäver"*)																												
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	Å	Ä	Ö
Gemener (*"små bokstäver"*)																												
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	å	ä	ö

Tyvärr är det svenska alfabetet inte den standar som världens datorer arbetar efter…

Nytt: Datatypen char

ASCII-tabell (American Standard Code for Information interchange, def 1960!)

b_4	b_3	b_2	b_1	Row ↓ / Column →	0 (0 0 0)	1 (0 0 1)	2 (0 1 0)	3 (0 1 1)	4 (1 0 0)	5 (1 0 1)	6 (1 1 0)	7 (1 1 1)
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: ???? ????     n     int
#1001: 0000 0001     i     int
#1010: ?
#1011: ?
#1100: 0101 0100     ch    char
```

Aha! Ett 'T'

För att datorn ska förstå att det inte är talet 0x54 (int) utan positionen för bokstaven 'T' används datatypen char, en bokstavskonstant skrivs inom ' '!

Nytt: Datatypen char

```c
/* Characters 0.1 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch1 = 'H';
    char ch2 = 'A';
    char ch3 = 'L';

    printf("In what movie could you see\n");
    printf("the %c%c%c9000 computer?\n",ch1,ch2,ch3);

    printf("CT Done.\n");
    return 0;
}
```

```
Characters Tester (CT)!
In what movie could you see
the HAL9000 computer?
CT Done.
```

%c används i printf och scanf för att hantera en (1) bokstav!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: 0100 1000     ch1   char
#1001: 0100 0001     ch2   char
#1010: 0100 1100     ch3   char
```

USASCII code chart

b7 b6 b5 / Bits	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b4 b3 b2 b1 / Row ↓ Column →	0	1	2	3	4	5	6	7
0 0 0 0	0 NUL	DLE	SP	0	@	P	`	p
0 0 0 1	1 SOH	DC1	!	1	A	Q	a	q
0 0 1 0	2 STX	DC2	"	2	B	R	b	r
0 0 1 1	3 ETX	DC3	#	3	C	S	c	s
0 1 0 0	4 EOT	DC4	$	4	D	T	d	t
0 1 0 1	5 ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6 ACK	SYN	&	6	F	V	f	v
0 1 1 1	7 BEL	ETB	'	7	G	W	g	w
1 0 0 0	8 BS	CAN	(	8	H	X	h	x
1 0 0 1	9 HT	EM	)	9	I	Y	i	y
1 0 1 0	10 LF	SUB	*	:	J	Z	j	z
1 0 1 1	11 VT	ESC	+	;	K	[	k	{
1 1 0 0	12 FF	FS	,	<	L	\	l	|
1 1 0 1	13 CR	GS	-	=	M	]	m	}
1 1 1 0	14 SO	RS	.	>	N	^	n	~
1 1 1 1	15 SI	US	/	?	O	_	o	DEL

Eftersom en char är en position i ASCII-alfabetet går det att behandla en char som en int, t.ex är 'A'=65!

Nytt: Datatypen char

USASCII code chart

b4	b3	b2	b1	Row ↓ / Column →	0 (000)	1 (001)	2 (010)	3 (011)	4 (100)	5 (101)	6 (110)	7 (111)
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

Otroligt viktigt!

Har du en char variabel som innehåller tecknet ’3’ kan du lätt få fram siffran 3 genom att subtrahera ’0’: 51-48=3!

```
int i='3'-'0';   // Initierar i till (talet) 3!
```

Har du en char variabel som innehåller en liten bokstav kan du lätt få samma stora bokstav genom att subtrahera ’a’ och därefter addera ’A’ (och vise versa)

```
ch=ch-'a'+'A';  // Liten till stor bokstav!
```

Nytt: Datatypen char

```c
/* Characters 0.3 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf("%c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
}
```

En sista sak…

```
Characters Tester (CT)!
Enter a letter :a
a [97]
Enter a letter :
 [10]
Enter a letter :A
A [65]
Enter a letter :
 [10]
Enter a letter :0
0 [48]
CT Done.
```

…?!

Nytt: Datatypen char

```c
/* Characters 0.3 AC */
#include <stdio.h>

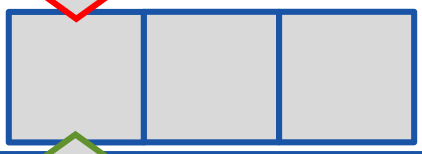
int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf("%c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
```

NEXT WRITE (Kontrolleras av OS:et!)



OS KEYB. INP. BUF.

NEXT READ (Styrs av programmet som vill läsa tecken!)

En sista sak…

```
Characters Tester (CT)!
Enter a letter :a
a [97]
Enter a letter :
 [10]
Enter a letter :A
A [65]
Enter a letter :
 [10]
Enter a letter :0
0 [48]
CT Done.
```

…?!

Programmet ”stannar” och väntar på input från tangentbordet då OS:ets tangentbordsbuffer är tom!

Nytt: Datatypen char

```c
/* Characters 0.3 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

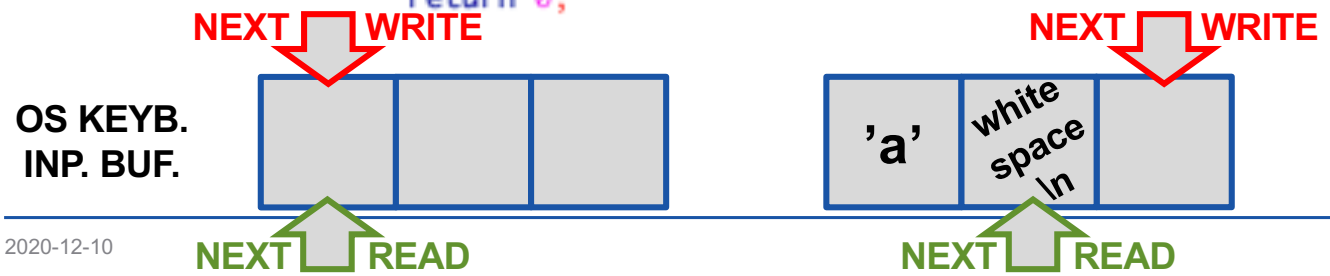
    do {
        printf("Enter a letter :");
        scanf("%c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
```

En sista sak…

```
Characters Tester (CT)!
Enter a letter :a
a [97]
Enter a letter :
 [10]
Enter a letter :A
A [65]
Enter a letter :
 [10]
Enter a letter :0
0 [48]
CT Done.
```

…?!



Användaren trycker på 'a' **och** return, scanf %c konsumerar 'a'!

Nytt: Datatypen char

```c
/* Characters 0.3 AC */
#include <stdio.h>

int main(void){

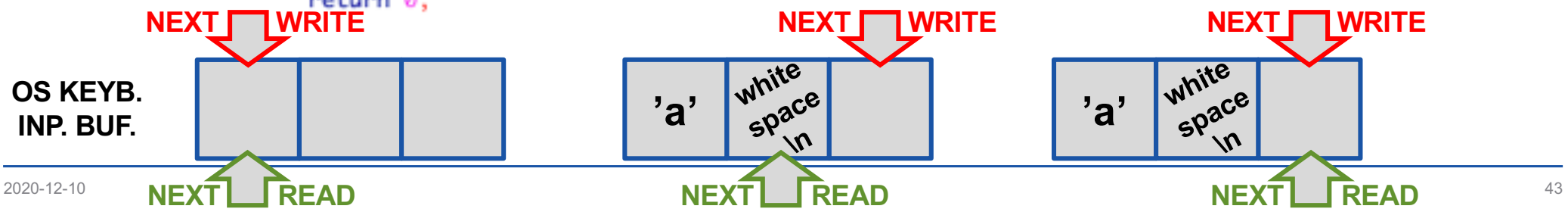
    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf("%c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
```

Return-tecknet finns kvar och konsumeras!

```
Characters Tester (CT)!
Enter a letter :a
a [97]
Enter a letter :
 [10]
Enter a letter :A
A [65]
Enter a letter :
 [10]
Enter a letter :0
0 [48]
CT Done.
```



Nytt: Datatypen char

```c
/* Characters 0.4 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf(" %c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
}
```

```
Characters Tester (CT)!
Enter a letter :b
b [98]
Enter a letter :B
B [66]
Enter a letter :0
0 [48]
CT Done.
```


Mellanslag i en scanf-matchningssträng betyder läs bort 0..n white spaces!

Nytt: Operatorn sizeof(type)

```c
/* Characters 0.6 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");

    printf("Size of char  : %zu\n", sizeof(char));
    printf("Size of int   : %zu\n", sizeof(int));
    printf("Size of float : %zu\n", sizeof(float));

    printf("CT Done.\n");
    return 0;
}
```

Talar om hur många Byte en datatyp kräver!

```
Characters Tester (CT)!
Size of char  : 1
Size of int   : 4
Size of float : 4
CT Done.
```

Används främst i samband med pekar (senare föreläsning) men finns med i kursen för att påtala ett problem som vi återkommer till!



Nytt: typedef

```
typedef int Euro;
Euro       wallet;

typedef unsigned int Quantity;
Quantity bults;
```

Wow, vi kan göra egna datatyper!

Otroligt viktigt lite längre fram…

…men bidrar till läsbarhet och att koden lättare kan flyttas mellan olika typer av maskiner!

Observera: När vi skapar en egen datatyp så ska namnet börja med stor bokstav!

Kommer du ihåg…

Läs in en bokstav, och skriv ut som stor, tills bokstaven var 0!

```
Characters Tester (CT)!
Enter a letter :a
A
Enter a letter :A
A
Enter a letter :0
0
CT Done.
```

1 **Skriv ner proble-met så att ett barn förstår det!**

Kommer du ihåg…

```
Characters Tester (CT)!
Enter a letter :a
A
Enter a letter :A
A
Enter a letter :0
0
CT Done.
```

Läs in en bokstav, och skriv ut som stor, tills bokstaven var 0!

1 **Skriv ner proble-met så att ett barn förstår det!**

2 **Gör ett minimalt main() av dina anteckningar!**

3 **Konstanter**
Variabler med värde
Uttryck & resultat!

4 **Läs in riktiga värden!**

```
// Programrubrik


// Repetera…
//    Skriv ut ledtext
//    Läs in bokstav*
//    om liten...
//       ...gör stor!
//    Skriv ut!
// ...så länge !'0'

// Klart!
```

* Undvik \n problemet!

Kommer du ihåg…

```
Characters Tester (CT)!
Enter a letter :a
A
Enter a letter :A
A
Enter a letter :0
0
CT Done.
```

Läs in en bokstav, och skriv ut som stor, tills bokstaven var 0!

1 **Skriv ner proble-met så att ett barn förstår det!**

2 **Gör ett minimalt main() av dina anteckningar!**

3 **Konstanter**
Variabler med värde
Uttryck & resultat!

4 **Läs in riktiga värden!**

```
// Programrubrik


// Repetera…
//    Skriv ut ledtext
//    Läs in bokstav*
//    om liten...
//       ...gör stor!
//    Skriv ut!
// ...så länge !'0'

// Klart!
```

* Undvik \n problemet!

```c
/* Characters 0.5 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf(" %c",&ch);
        if ('a'<=ch && ch<='z')
            ch=ch-'a'+'A';
        printf("%c\n",ch);
    } while (ch!='0');

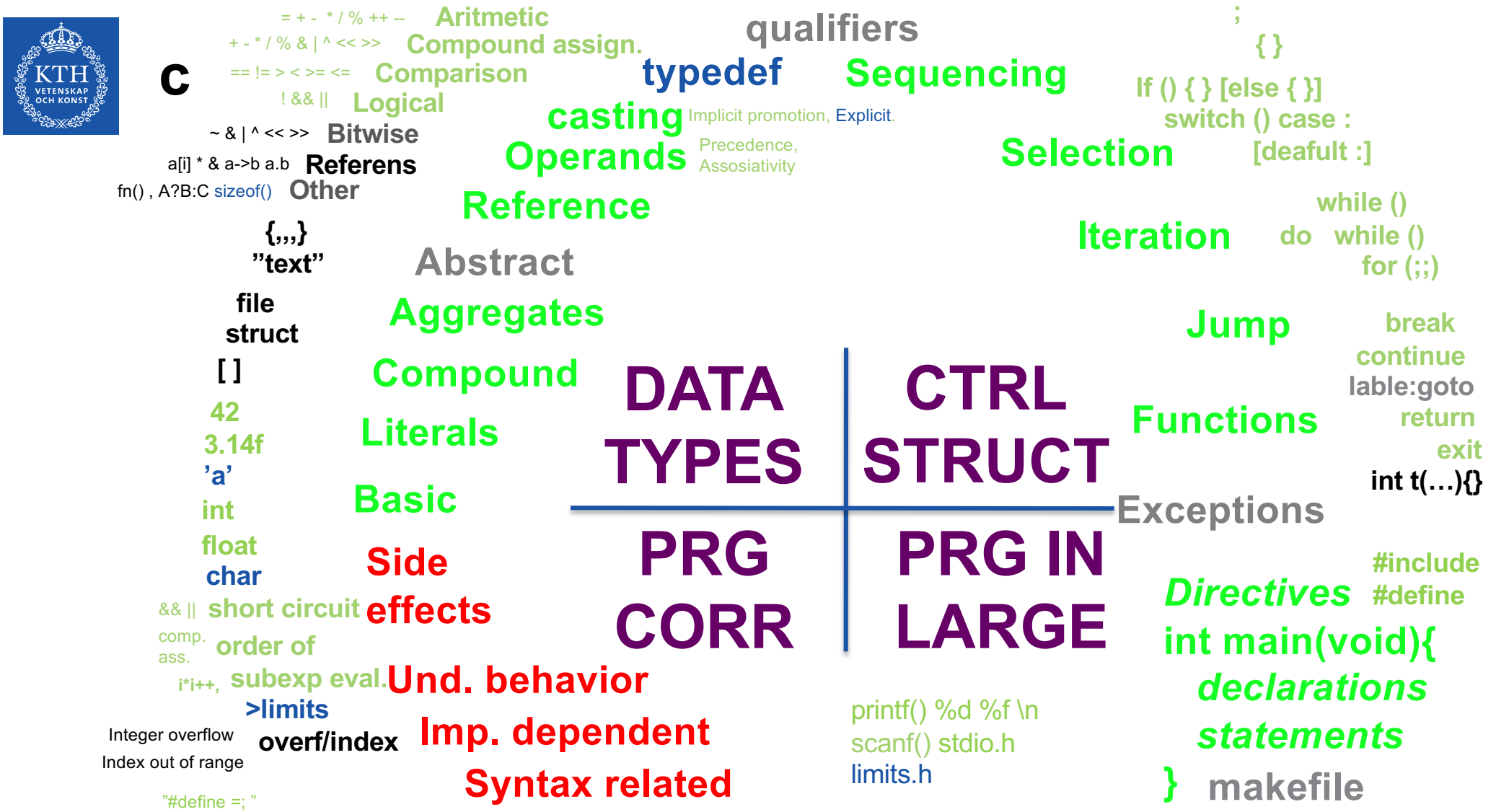
    printf("CT Done.\n");
    return 0;
}
```



Ska ”sitta” till nästa gång (Etapp #1 av 3)…

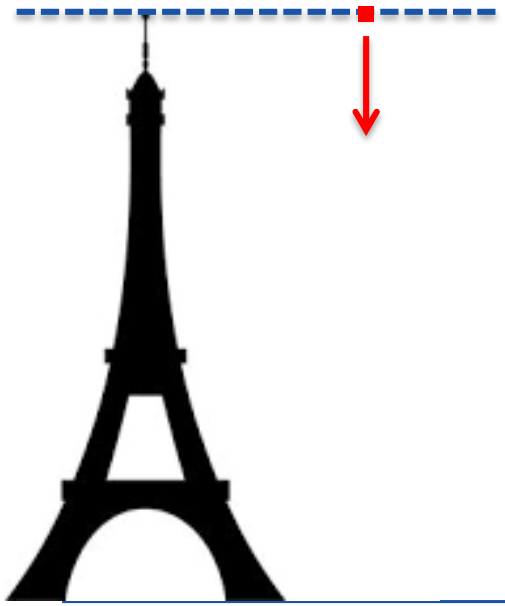
```
#include <bibliotek>
#define KONSTANTER tal

int main(void) {                          // main funktionen
   int a=1;                               // heltalsvariabler
   float b=1.0f;                          // flyttalsvariabler
   char c=’c’;                            // teckenvariabler
   if (a) c=’d’; else {c=’e’;}            // if och sammansatt sats
   for (int i=0; i<a;i++)c++;             // for/while/do while
   switch (c) {default : a++;}            // switch/case/break/default
}                                         // INDENTERA RÄTT
```


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
Reference
Abstract
Aggregates
Compound
Literals
Basic
Side effects
Und. behavior
Imp. dependent
Syntax related
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
Iteration
do while ()
for (;;)
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
#include
#define
Directives
int main(void){
declarations
statements
} makefile
printf() %d %f \n
scanf() stdio.h
limits.h
int/int
/*comment*/ or //

Lab #1…

h=324; g=10
t=0; s=0



Uppgift: Tabell för v & s för varje sekund tills bollen träffar marken!

$$v=at \;\&\; s=0.5at^2$$

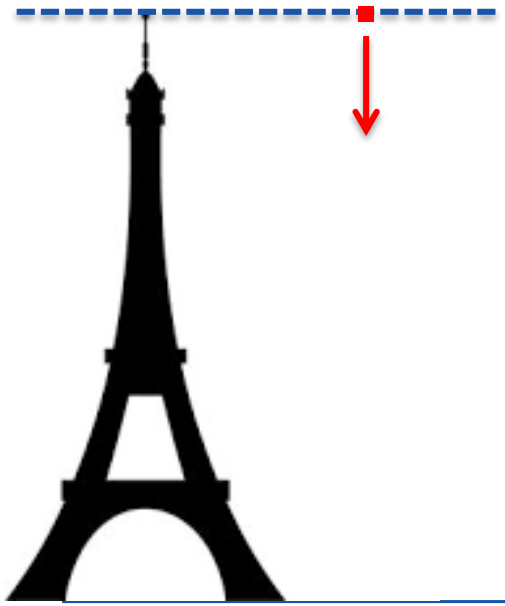
```
#0000: ?
#0001: ?     HUR HADE DU
#0010: ?     GJORT DET HÄR
#0011: ?     FÖR HAND
#0100: ?
#0101: ?
#0110: ?
#0111: ?
#1000: ?
#1001: ?
#1010: ?
#1011: ?



 VART   VÄRDE.       NAMN TYP
#1100: ?              ?    ?
#1101: ?              ?    ?
#1110: ?              ?    ?
#1111: ?              ?    ?
```

Lab #1…

h=324; g=10
t=0; s=0



$v=at$ &
$s=0.5at^2$

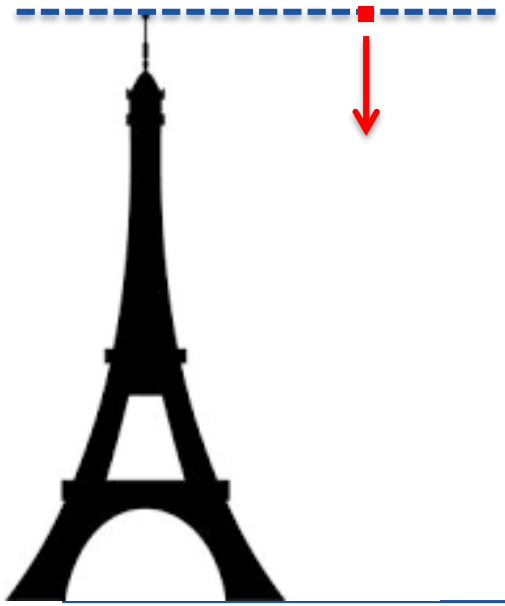
```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: ?
#0100: visa/rita t, s & v
#0101: ?
#0110: ?
#0111: ?
#1000: ?
#1001: Visa klart!
#1010: ?
#1011: ?



 VART  VÄRDE.      NAMN TYP
#1100: ?             ?   ?
#1101: ?             ?   ?
#1110: ?             ?   ?
#1111: ?             ?   ?
```

Lab #1…

h=324; g=10
t=0; s=0



$v=at$ &
$s=0.5at^2$

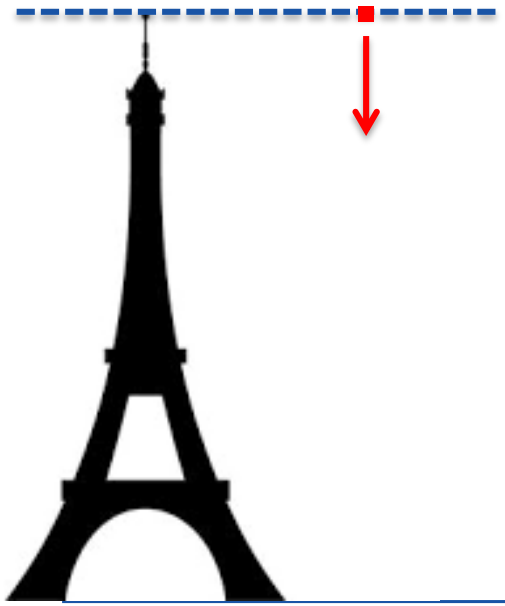
```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: ?
#0100: visa/rita t, s & v
#0101: t = t + 1
#0110: v = 10 * t
#0111: s = 0.5 * 10 * t * t
#1000: ?
#1001: Visa klart!
#1010: ?
#1011: ?



 VART  VÄRDE.      NAMN TYP
#1100: ?             ?   ?
#1101: ?             ?   ?
#1110: ?             ?   ?
#1111: ?             ?   ?
```

Lab #1…

h=324; g=10
t=0; s=0



Vi provkör!

Vad händer om
vi utför rad 0?

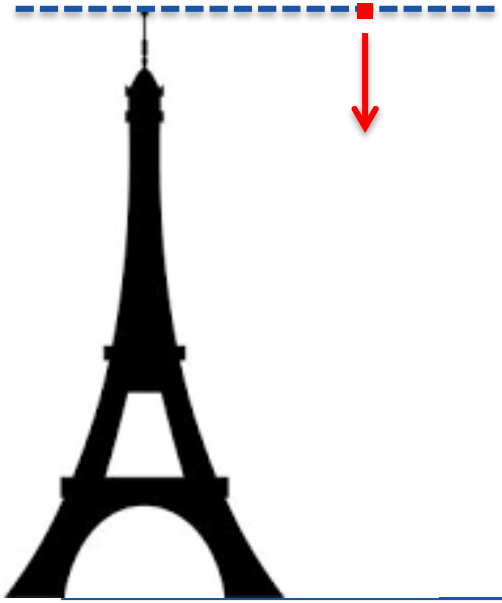
```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

VART	VÄRDE.	NAMN	TYP
#1100:	?	?	?
#1101:	?	?	?
#1110:	?	?	?
#1111:	?	?	?

Lab #1…

h=324; g=10

t=0; s=0



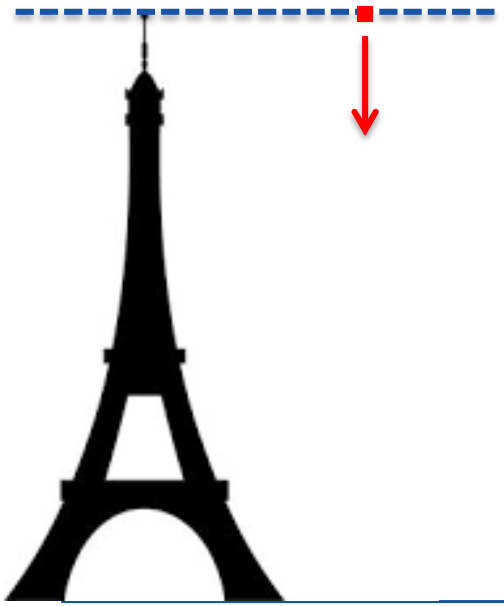


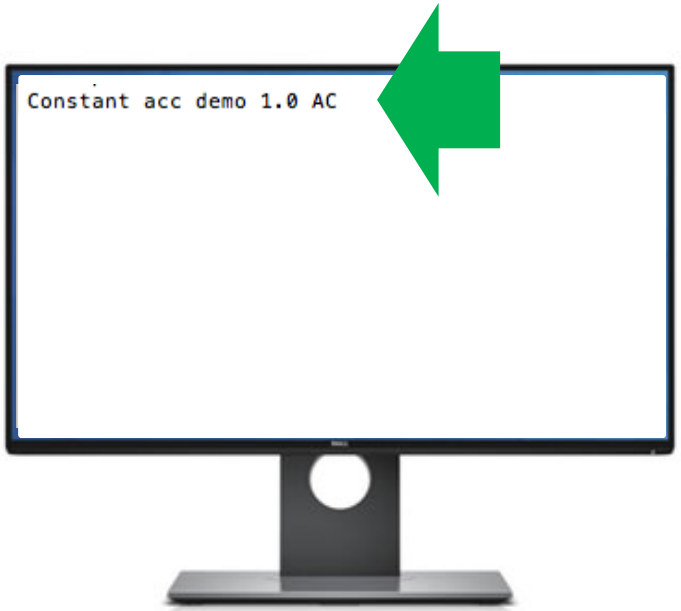
```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

VART	VÄRDE.	NAMN	TYP
#1100:	0	t	heltal
#1101:	0	v	heltal
#1110:	0	s	heltal
#1111:	?	?	?

Lab #1…

h=324; g=10
t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?



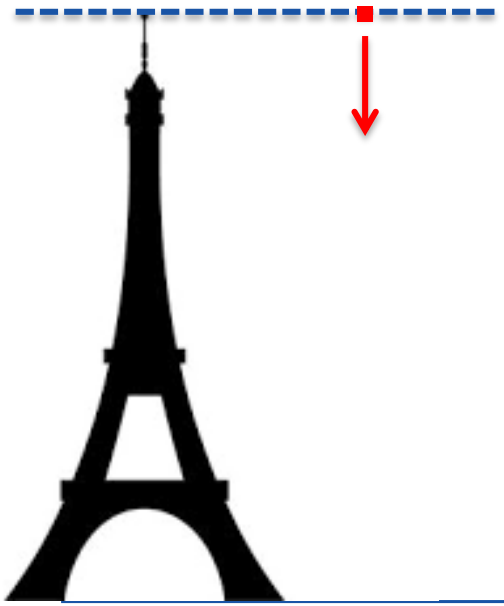
 VART  VÄRDE.      NAMN TYP
#1100: 0             t   heltal
#1101: 0             v   heltal
#1110: 0             s   heltal
#1111: ?             ?   ?
```

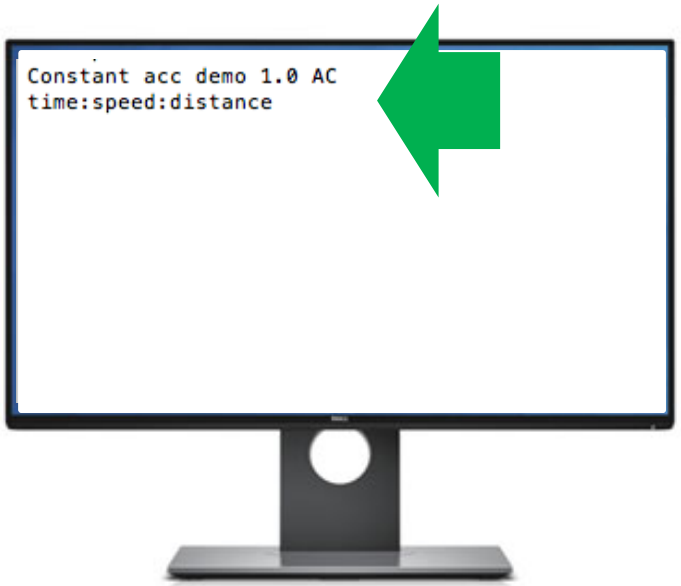

Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?



 VART  VÄRDE.      NAMN TYP
#1100: 0             t   heltal
#1101: 0             v   heltal
#1110: 0             s   heltal
#1111: ?             ?   ?
```

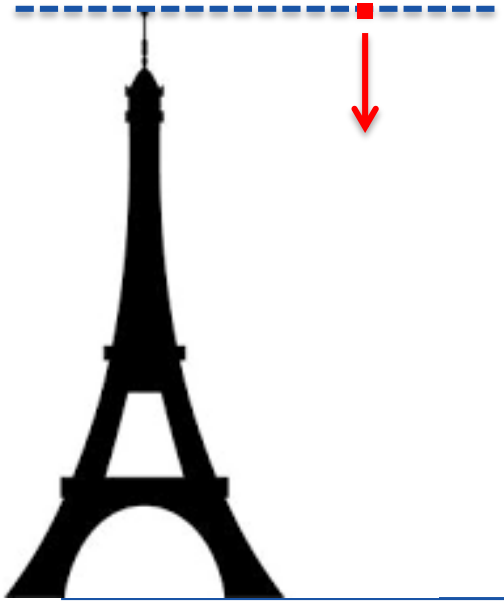

Bra början, vad innebär nästa rad?

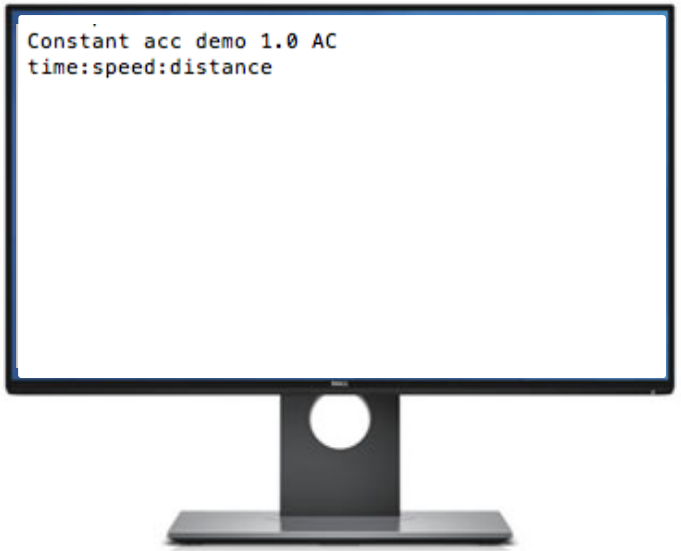
Lab #1…

Ingenting, kom bara ihåg den här platsen!

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

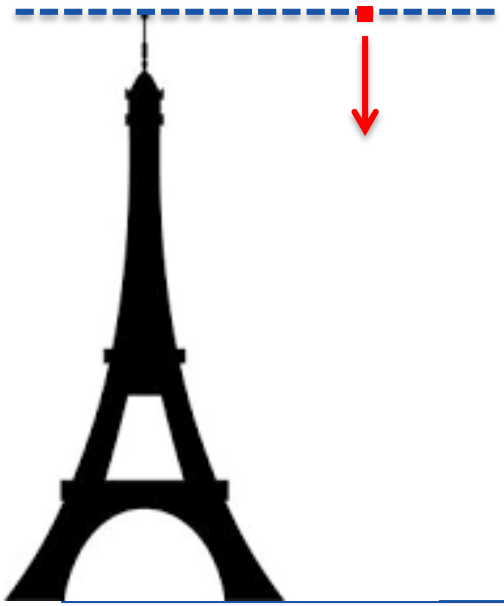
VART	VÄRDE.	NAMN	TYP
#1100:	0	t	heltal
#1101:	0	v	heltal
#1110:	0	s	heltal
#1111:	?	?	?

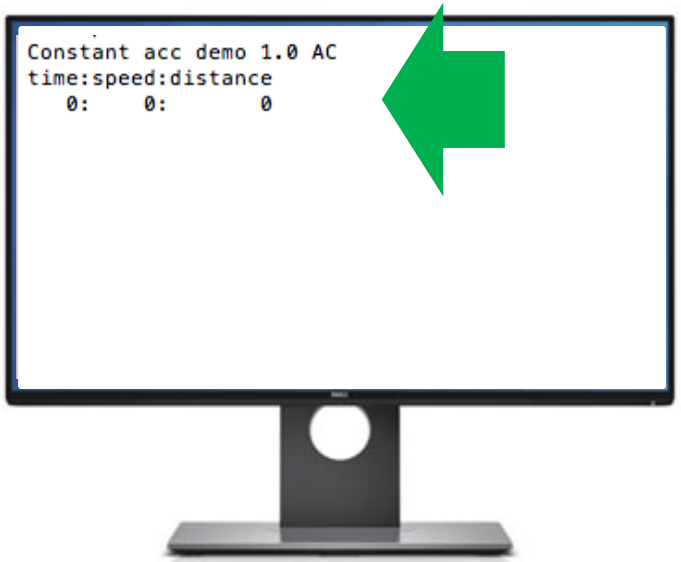
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

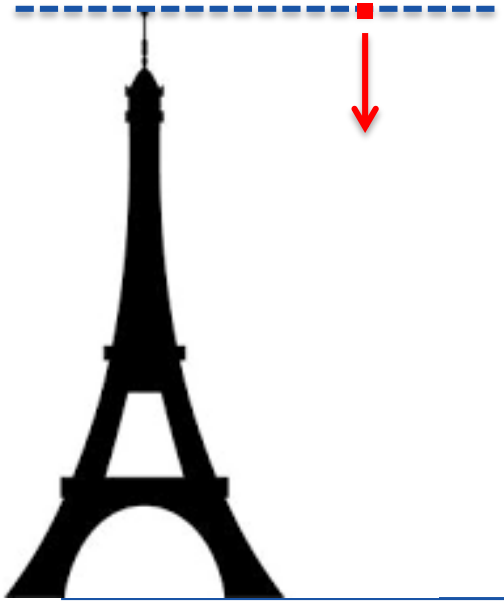
VART	VÄRDE.	NAMN	TYP
#1100:	0	t	heltal
#1101:	0	v	heltal
#1110:	0	s	heltal
#1111:	?	?	?

Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

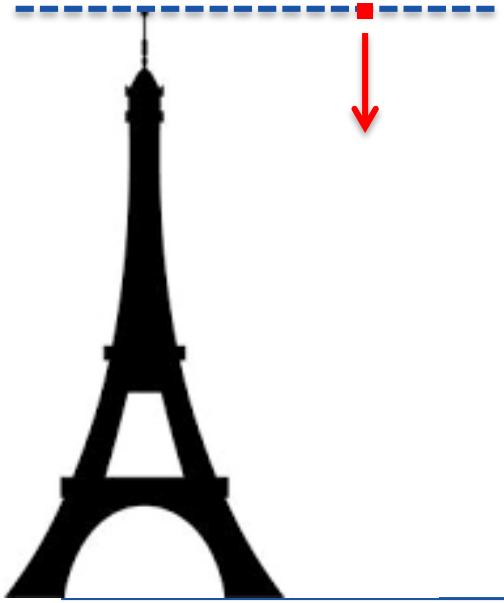
VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	0	v	heltal
#1110:	0	s	heltal
#1111:	?	?	?

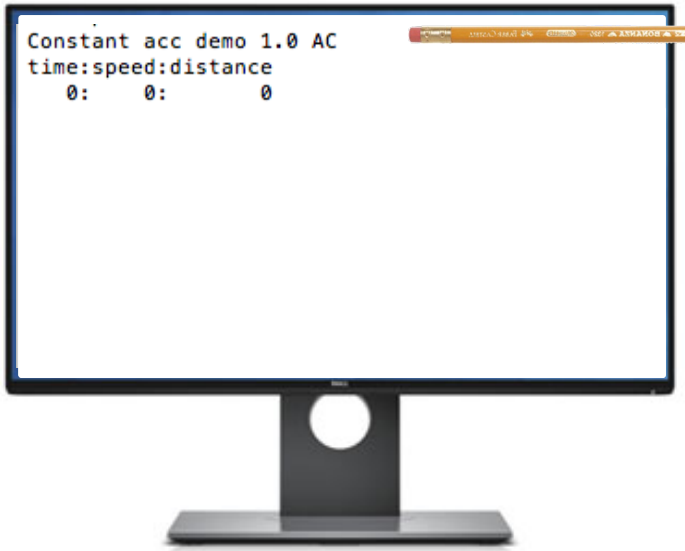
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

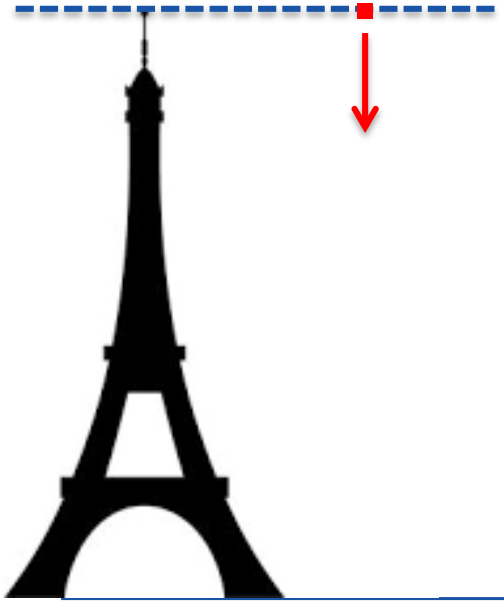
VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	10	v	heltal
#1110:	0	s	heltal
#1111:	?	?	?

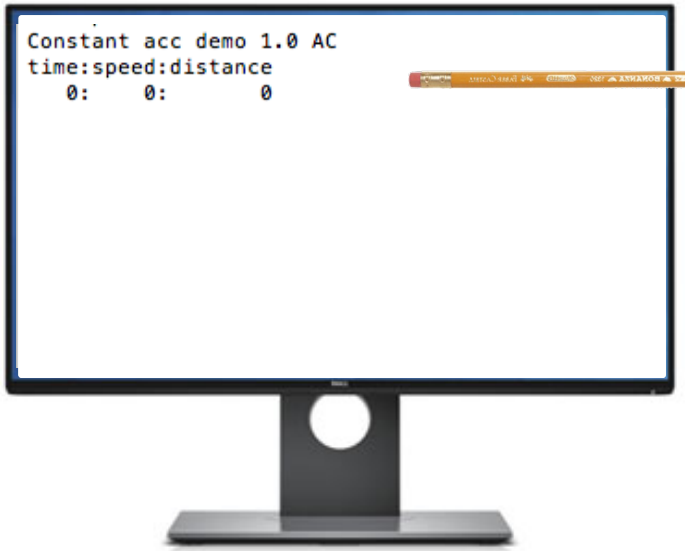
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	10	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

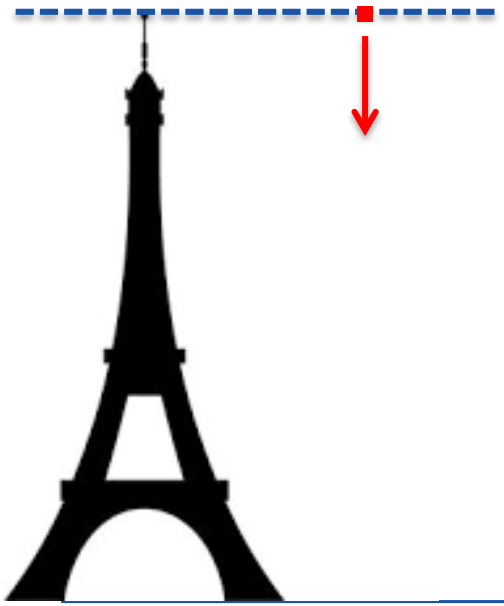
Bra början, vad innebär nästa rad?

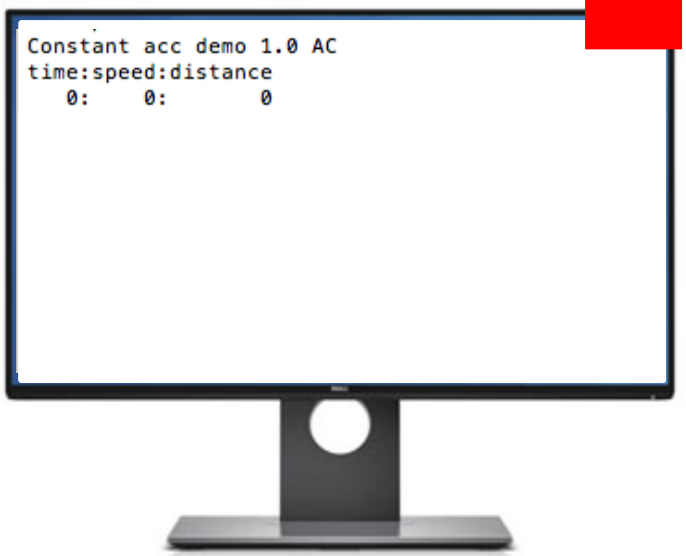
Lab #1…

Ingenting, kom bara ihåg den här platsen!

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

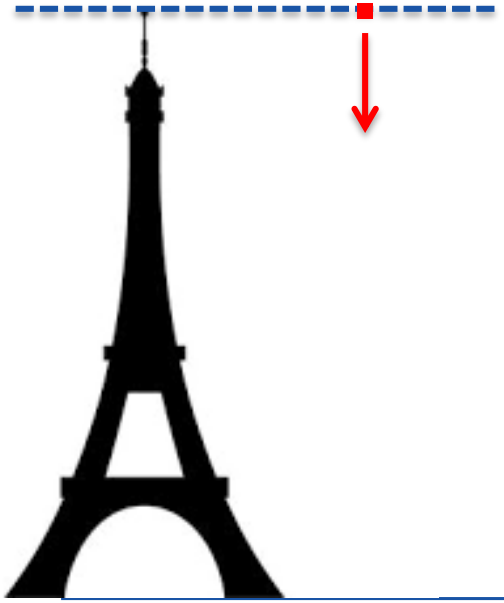
VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	10	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

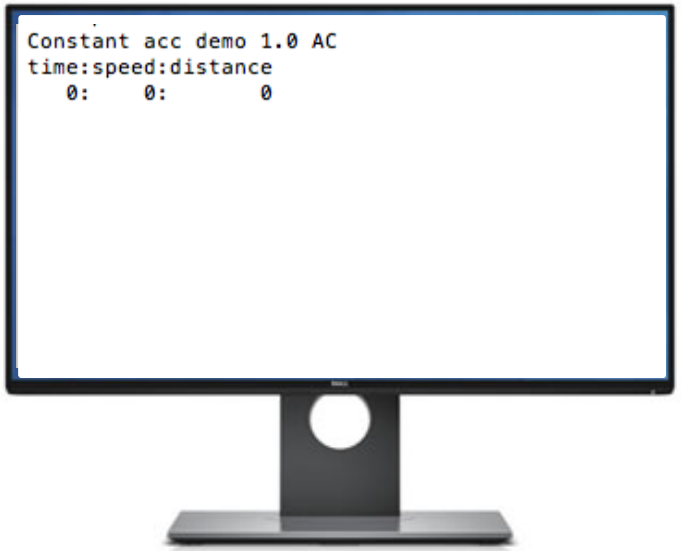
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

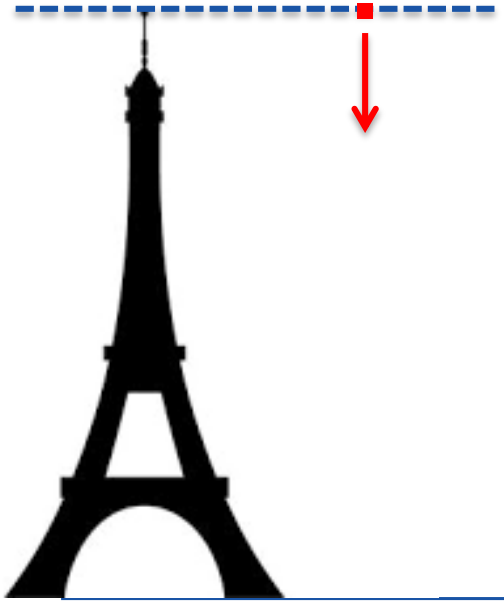
VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	10	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

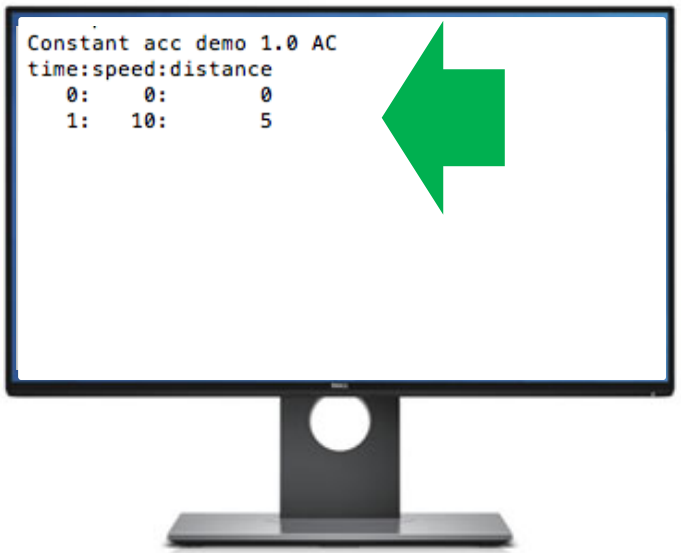
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

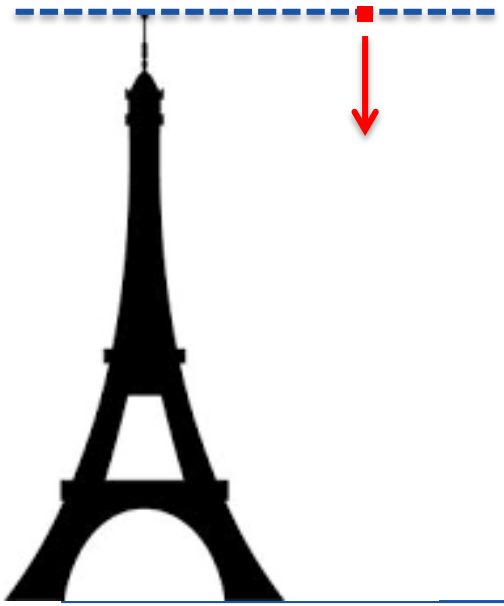
VART	VÄRDE.	NAMN	TYP
#1100:	1	t	heltal
#1101:	10	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

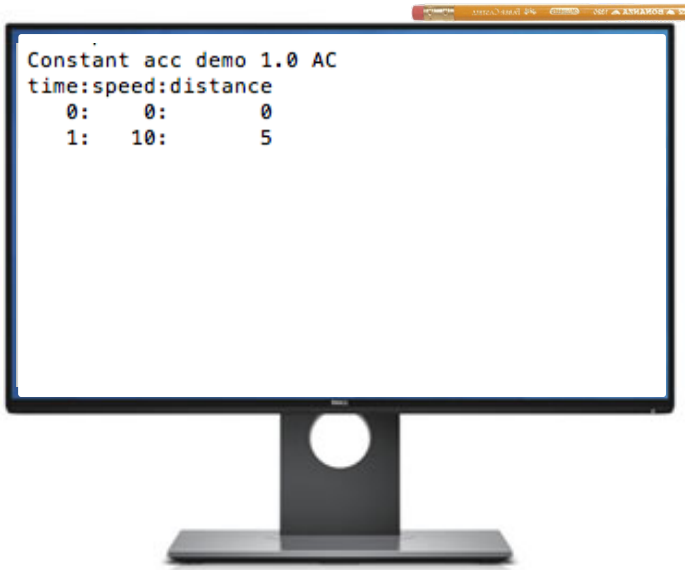
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

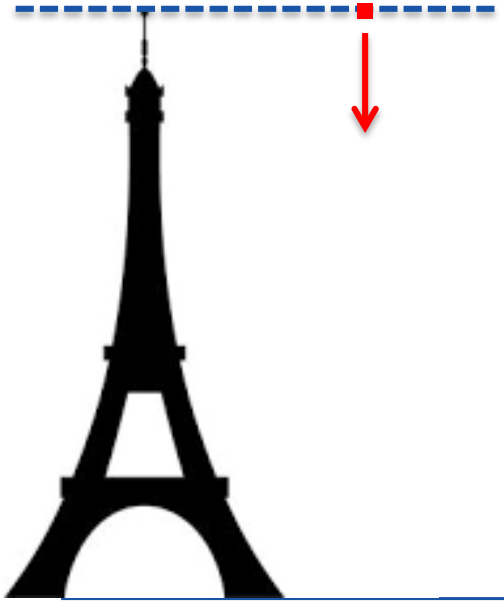
VART	VÄRDE.	NAMN	TYP
#1100:	2	t	heltal
#1101:	10	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

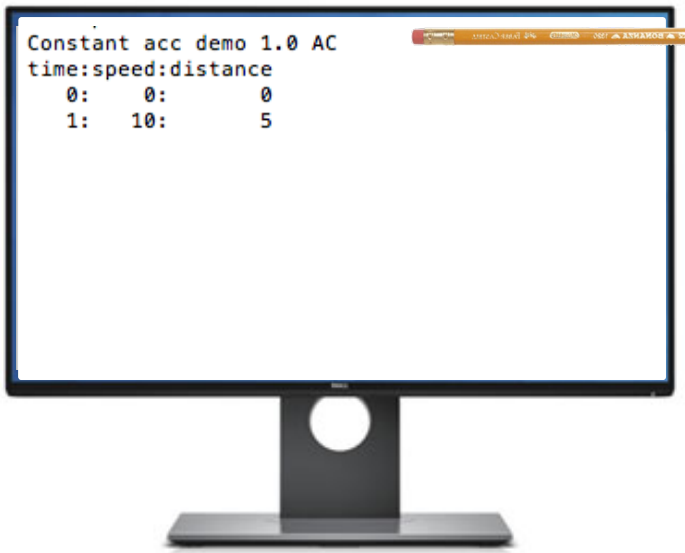
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

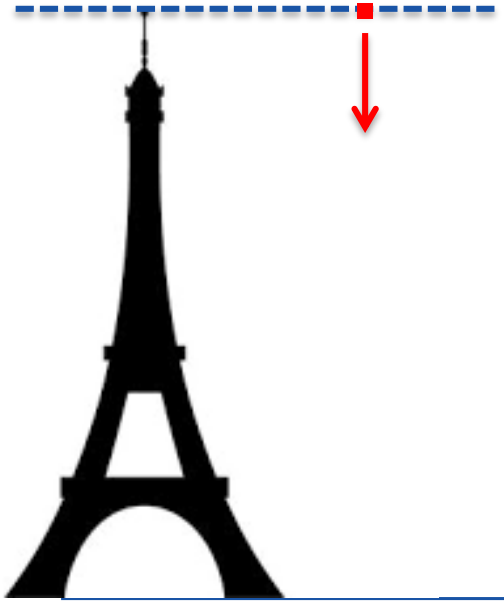
VART	VÄRDE.	NAMN	TYP
#1100:	2	t	heltal
#1101:	20	v	heltal
#1110:	5	s	heltal
#1111:	?	?	?

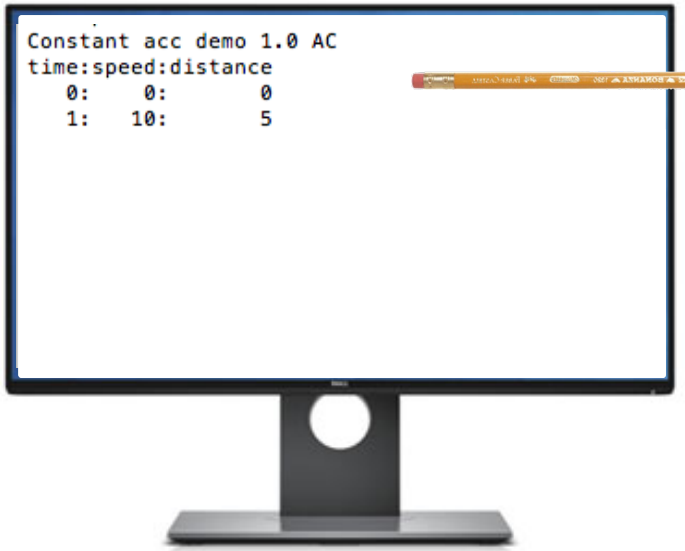
Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0






```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

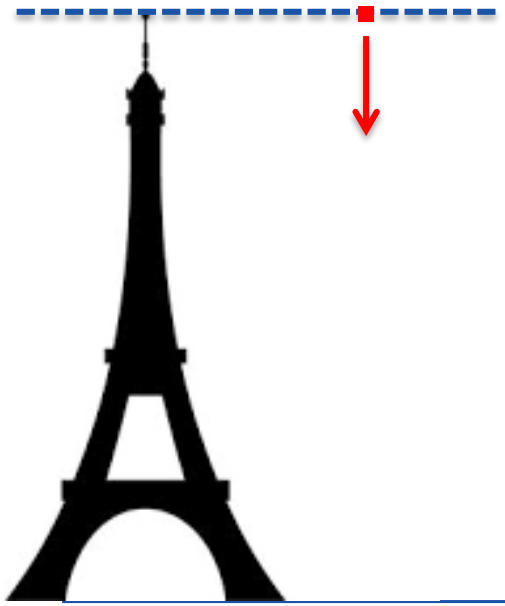
VART	VÄRDE.	NAMN	TYP
#1100:	2	t	heltal
#1101:	20	v	heltal
#1110:	20	s	heltal
#1111:	?	?	?

Bra början, vad innebär nästa rad?

Lab #1…

h=324; g=10

t=0; s=0



```
Constant acc demo 1.0 AC
time:speed:distance
   0:    0:       0
   1:   10:       5
   2:   20:      20
   3:   30:      45
   4:   40:      80
   5:   50:     125
   6:   60:     180
   7:   70:     245
   8:   80:     320
Ground [8..9] sek!
Program completed, bye!
```

```
#0000: t=0, s=0, v=0
#0001: Visa/rita programtitel
#0010: Visa/rita tabellrubrik
#0011: upprepa…
#0100:    visa/rita t, s & v
#0101:    t = t + 1
#0110:    v = 10 * t
#0111:    s = 0.5 * 10 * t * t
#1000: …så länge s < 324
#1001: Visa klart!
#1010: ?
#1011: ?
```

VART	VÄRDE.	NAMN	TYP
#1100:	9	t	heltal
#1101:	90	v	heltal
#1110:	405	s	heltal
#1111:	?	?	?

Wow, ett fungerande program!

Lab #1…

”Skissa” på en lösning!

```
t=0, s=0, v=0
Visa/rita programtitel
Visa/rita tabellrubrik
upprepa…
    visa/rita t, s & v
    t = t + 1
    v = 10 * t
    s = 0.5 * 10 * t * t
…så länge s < 324
Visa/rita klart!
```

Skriv in den i datorn!

```c
#include <stdio.h>
int main(int argc, char **argv)
{
    int t=0,s=0,v=0;
    printf("Constant acc demo 1.0 AC\n");
    printf("time:speed:distance\n");
    do {
        printf("%4d:%5d:%8d\n",t,v,s);
        t=t+1;
        v=10*t;
        s=0.5*10*t*t;
    } while (s<324);
    printf("Ground [%d..%d] sek!\n",t-1,t);
    printf("Program completed, bye!\n");
    return 0;
}
```

Kompilera

Provkör!

```
#0000: 1000 0000 0101 0000
#0001: o.s.v
#0010:
#0011:
#0100:
#0101:
#0110:
#0111:
#1000:
#1001
#1010
#1011
#1100
#1101
#1110
#1111:
```

```
Constant acc demo 1.0 AC
time:speed:distance
   0:    0:       0
   1:   10:       5
   2:   20:      20
   3:   30:      45
   4:   40:      80
   5:   50:     125
   6:   60:     180
   7:   70:     245
   8:   80:     320
Ground [8..9] sek!
Program completed, bye!
```

I små steg

& provkör varje steg!

Övn #5: Kapitel 7:P4, P10 & P11

2=ABC
3=DEF
4=GHI
5=JKL
6=MNO
7=PRS
8=TUV
9=WXY
Otherwize unchanged
All letters uppercase

1-800-COLLECT
1-800-2655328

Laboration #1

A e i o u
And that's the way it is.
Your sentence contains 6 vowels.

Lloyd Fosdick
Fosdick, L.

K7.P12: 1+2*3

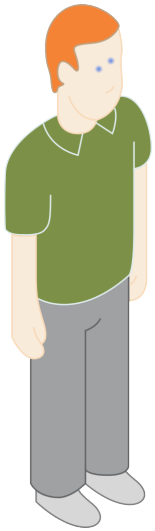
1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!
a..n

4 Läs in riktiga värden!

Historik



• 150710 0.1	AC	Skapad.
• 150711 0.2	AC	Range
• 150712 0.5	AC	Char
• 150714 0.9	AC	cast
• 150715 1.0	AC	typedef
• 160810 1.1	AC	Mindre uppdateringar
• 170904 1.2	AC	Lagt till övningen Eiffeltornet
• 180910 1.3	AC	Mindre uppdateringar
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.